Yeahbot

松 甲 科 技

PS2 手柄说明手册

杭州松甲科技有限公司

手册版本 **V1.0**

➢ 前提说明

本手册为松甲科技出品的　手册。温馨提示，使用手柄前，请仔细阅读该手册，以免盲目使用导致控制板损坏。

用户收到产品时请及时对照发货清单检查配件是否齐全，以及快递运输过程中产品是否损坏等现象，如有问题请及时联系淘宝客服人员（注：自收到货起三日内没有反应问题，视为产品收货正常）。

学习资料将以百度云网盘链接形式发送，视频资料可在 **B** 站进行观看学习。

散件组装接线时请按照教程进行组装接线，因操作不正确导致产品损坏的，一切后果由自己承担！如有问题请及时联系客服或售后人员。我们提供全程有关产品技术支持（可通过电话/微信等）。

售后时间：10：00---21：00（周一到周六）

淘宝店铺	首页-松甲科技机器人商城-淘宝网（taobao.com）
松甲 B 站	松甲科技
售后途径	淘宝旺旺/电话/微信群等
	电话：0571-86222021/18758140296/15395826513
	微信群：淘宝联系我们客服，拉进对应的群聊。

➢ 版权申明

本文由杭州松甲科技编写，受版权保护。未经许可，禁止以任何形式进行复制、修改、发布、传播或用于商业用途。

未经杭州松甲科技明确授权，禁止以任何形式复制、修改、发布、传播或用于商业用途。这包括但不限于将本文用于商业产品、销售、以及其他商业活动。

其他权利保留：杭州松甲科技保留有权对本版权说明进行修改和更新的权利。对于未涉及到的使用情况和授权请求，杭州松甲科技保留最终解释权。

目录

1. 手柄说明 **3**
1.1. 手柄介绍 3
1.2. 转接板介绍 4
1.3. 接收器说明 6
1.4. 配对说明 1
1.5. 协议说明 1
2. STM32 例程说明 **3**
2.1. 硬件连接 3
2.2. 程序说明 3
3. Arduino 例程说明 **10**
3.1. 硬件连接 10
3.2. 程序说明 10
4. 51 单片机例程说明 **11**
5. FAQ 问题 **11**

1.手柄说明

索尼的 PS2 无线手柄是索尼游戏机的遥控手柄，控制协议已被全球创客破解，可以方便用于 机器人控制器，作为遥控输入设备。这款手柄价格便宜，性价比极高。

1.1. 手柄介绍

手柄按键定义如下图，具有 2 个 360°摇杆和 17 个物理按键，摇杆也可以作为按键使用。

手柄通过 2 节 7 号电池供电，使用前请安装电池。并将下图所示开关打开，拨动到 ON 一侧 为打开。

手柄采用 2.4G 无线技术，遥控距离可达 15 米，信号更强更稳定，特别适合机器人、智能小车、机械臂、无人机等场景。

无配对状态下，30s 后启动休眠模式；开启并配对完成后，5 分钟无按键按下，自动启动休眠 模式。按下“START”即可唤醒手柄。

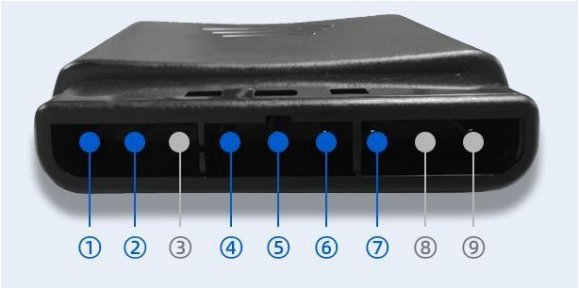
1.2. 转接板介绍

市面上PS2手柄工作电压均为3.3V，虽然使用5V供电也能工作，一旦连接舵机之类感性负载，手柄必定会烧坏。本转接板做了专门保护的设计。

数据接口均加有保护电阻，兼容 3.3V 和 5V 单片机。电源输入带有反接保护电路，有效保护手柄、控制器不被损坏。

多种插座型号可选，满足用户差异化需求。

无线手柄通讯一共需要用到6个引脚，其中③⑧为空引脚未使用。

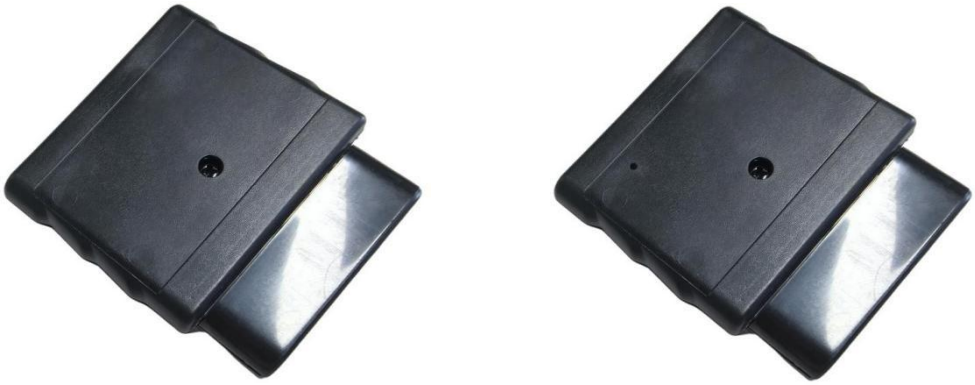


引脚	定义	引脚说明
①	DI/DAT	信号从手柄到手机，8bit串行数据，在时钟的下降沿进行传送。读取信号需要在时钟由高到低的变化过程中进行
②	DO/CMD	信号从主机到手柄，8bit串行数据，也是在时钟的下降沿进行传送，此信号与DI/DAT信号相对应
③	GND	手柄电源地信号
⑤	VDD	接收器工作电源，供电电压范围3～5V。
⑥	CS/SEL	手柄工作使能信号，在通信期间，设置为低电平
⑧	CLK	时钟信号，由主机提供，用于保持数据同步

接口处有清晰的引脚定义丝印标识，方便用户接线。

1.3. 接收器说明

无线接收器分为普通款和锁定款，锁定款可与任一手柄锁定，锁定后该接收器只能与锁定的手柄进行配对，锁定款背部有小孔，内置一个用于配对手柄的按钮，其他与普通款外观一致。



手柄接收器实物图

1.4. 配对说明

- 普通款接收器

手柄电源打开后，在未搜索到接收器的状况下，手柄的红灯和绿灯会快速闪烁，进入等待配对状态。

如果手柄接收器已经连接了单片机控制信号，并上电运行手柄程序。手柄未开启状态下，接收器电源灯红灯常亮，信号灯绿灯快速闪烁，进入等待配对状态。此时开启手柄后，手柄和接收器配对成功后，手柄和接收器的指示灯会常亮显示。

- 锁定款接收器

手柄电源打开后，在未搜索到接收器的状况下，手柄的红灯和绿灯会快速闪烁，进入等待配对状态。

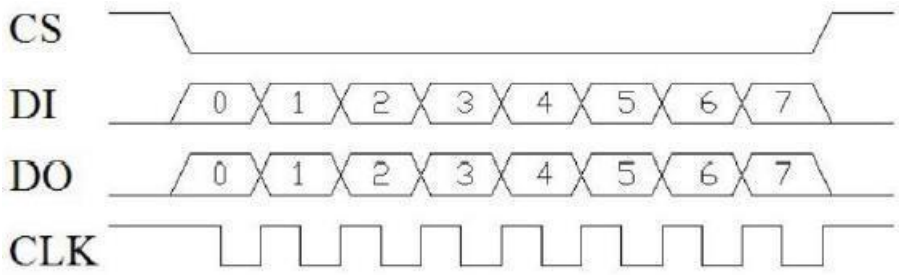
如果手柄接收器已经连接了单片机控制信号，并上电运行手柄程序。手柄未开启状态或手柄未与接收器锁定，接收器电源灯绿灯常亮，信号灯红灯慢速闪烁，进入等待配对状态。此时开启手柄后，手柄和接收器锁定并配对成功后，手柄和接收器的指示灯会常亮显示。

按下接收器背部小孔中的按钮，接收器红灯开始快速闪烁，打开手柄开关，红灯不再闪烁说明手柄与接收器锁定成功，锁定款手柄出厂默认已与配套手柄锁定

1.5. 协议说明

PS 手柄信号，所有通讯都是8 bit 串行数据最低有效位先行。在 PS 手柄总线的所有时码在时 钟下降沿都是同步的。传送一个字节的情况如下所示。数据线的逻辑电平在时钟下降沿驱动下触发改变。数据的接收读取在时钟的前沿（在记号*处）到电平变化之前完成。 在被选手柄接收每个 COMMAND 信号之后，手

柄需拉低 ACK 电平在最后一个时钟。如果被选手柄没 ACK 应答主机 将假定没手柄接入。



当 PS2 主机想读一个手柄的数据时，将会拉低 ATT 线电平并发出一个开始命令 (0x01)。手柄 将会回复它的 ID (0x41=数字, 0x23=NegCon, 0x73=工作模式). 在手柄发送 ID 字节的同时主机 将传送 0x42 请求数据。随后命令线将空闲和手柄送出 0x5A 意思说：“数据来了”。

字节定义如下。左右摇杆模拟值为无效，推到极限时，对应发送 UP、 RIGHT、 DOWN、 LEFT、△、○、 X、□, 按键 L3、 R3 无效。

BYTE	CMND	DATA
01	0x01	idle
02	0x42	0x73
03	idle	0x5A Bit0 Bit1 Bit2 Bit3 Bit4 Bit5 Bit6 Bit7
04	idle	data SLCT JOYR JOYL STRT UP RGHT DOWN LEFT
05	idle	data L2 R2 L1 R1 /\ O X |_|
06	idle	data Right Joy 0x00 = Left 0xFF = Right
07	idle	data Right Joy 0x00 = Up 0xFF = Down
08	idle	data Left Joy 0x00 = Left 0xFF = Right
09	idle	data Left Joy 0x00 = Up 0xFF = Down

2.STM32 例程说明

本小节讲解通过 STM32 连接手柄的使用，资料里面提供三个版本的代码，这里以 STM32F103C8T6 为例讲解

2.1. 硬件连接

例程硬件连接关系如下。

DI->PB4；

DO->PB5；

CS->PB6；

CLK->PB7。

2.2. 程序说明

完整程序详见工程文件。 这里主要介绍 ps2.c 文件中的函数。

端口初始化

```
void ps2_init(void)
{
    GPIO_InitTypeDef GPIO_InitStructure;
    RCC_APB2PeriphClockCmd(PS2_DAT_GPIO_CLK | PS2_CMD_GPIO_CLK | PS2_CS_GPIO_CLK | PS2_CLK
_GPIO_CLK, ENABLE); // 使能端口时钟
    GPIO_InitStructure.GPIO_Pin = PS2_DAT_PIN;          // 端口配置
    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_IPD;       // 下拉输入
    GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;  // 50M
    GPIO_Init(PS2_DAT_GPIO_PORT, &GPIO_InitStructure); // 根据设定参数初始化GPIO
    GPIO_InitStructure.GPIO_Pin = PS2_CMD_PIN;          // 端口配置
```

```
    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_Out_PP;    // 推挽输出
    GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;  // 50M
    GPIO_Init(PS2_CMD_GPIO_PORT, &GPIO_InitStructure); // 根据设定参数初始化GPIO
    GPIO_InitStructure.GPIO_Pin = PS2_CS_PIN;          // 端口配置
    GPIO_Init(PS2_CS_GPIO_PORT, &GPIO_InitStructure); // 根据设定参数初始化GPIO
    GPIO_InitStructure.GPIO_Pin = PS2_CLK_PIN;          // 端口配置
    GPIO_Init(PS2_CLK_GPIO_PORT, &GPIO_InitStructure); // 根据设定参数初始化GPIO
    PS2_CS(1);
    PS2_CLK(1);
    PS2_CMD(1);
}
```

PS2 解码通讯

```
void ps2_write_read(void)
{
    u8 i;
    PS2_CS(0);
    ps2_buf[0] = ps2_transfer(START_CMD);
    ps2_buf[1] = ps2_transfer(ASK_DAT_CMD);
    ps2_buf[2] = ps2_transfer(ps2_buf[0]);
    ps2_buf[3] = ps2_transfer(ps2_buf[0]);
    ps2_buf[4] = ps2_transfer(ps2_buf[0]);
    ps2_buf[5] = ps2_transfer(ps2_buf[0]);
    ps2_buf[6] = ps2_transfer(ps2_buf[0]);
    ps2_buf[7] = ps2_transfer(ps2_buf[0]);
    ps2_buf[8] = ps2_transfer(ps2_buf[0]);
    PS2_CS(1);
    /* PS2数据解析*/
    if ((ps2_buf[0] == 255) && (ps2_buf[2] == 90)) /* 判断ps2是否连接上*/
```

```
{
    ps2_data.isConnected = 1;
}
else
{
    ps2_data.isConnected = 0;
}
if (ps2_data.isConnected) /* 连接成功才运行下面数据解析*/
{
    if (ps2_buf[1] == PS2_MODE_GRN) /* 判断PS2模式 */
    {
        ps2_data.mode = 0;
    }
    else
    {
        ps2_data.mode = 1;
    }
    if (ps2_data.mode) /* 为红灯模式读取解析数据*/
    {
        /* 保存上一次数据*/
        ps2_data.last_left_x = ps2_data.left_x;
        ps2_data.last_left_y = ps2_data.left_y;
        ps2_data.last_right_x = ps2_data.right_x;
        ps2_data.last_right_y = ps2_data.right_y;
        /* 读取数据 */
        ps2_data.right_x = (ps2_buf[5] - 0x7f);
        ps2_data.right_y = (ps2_buf[6] - 0x80);
        ps2_data.left_x = (ps2_buf[7] - 0x7f);
```

```
            ps2_data.left_y = (ps2_buf[8] - 0x80);

        }

        /* 获取左边按键数据*/

        ps2_data.left_up = !(ps2_buf[3] & 0x10);

        ps2_data.left_right = !(ps2_buf[3] & 0x20);

        ps2_data.left_down = !(ps2_buf[3] & 0x40);

        ps2_data.left_left = !(ps2_buf[3] & 0x80);

        /* 获取右边按键数据*/

        ps2_data.right_up = !(ps2_buf[4] & 0x10);

        ps2_data.right_right = !(ps2_buf[4] & 0x20);

        ps2_data.right_left = !(ps2_buf[4] & 0x80);

        ps2_data.right_down = !(ps2_buf[4] & 0x40);

        /* 获取前面左右各两个按键*/

        ps2_data.left_1 = !(ps2_buf[4] & 0x04);

        ps2_data.left_2 = !(ps2_buf[4] & 0x01);

        ps2_data.right_1 = !(ps2_buf[4] & 0x08);

        ps2_data.right_2 = !(ps2_buf[4] & 0x02);

        /* 获取中间两个按钮*/

        ps2_data.select = !(ps2_buf[3] & 0x01);

        ps2_data.start = !(ps2_buf[3] & 0x08);

        /* 清空数据数组值，方便下次解析数据 */

        for (i = 0; i <= 8; i++)

        {

            ps2_buf[i] = 0;

        }

    }
```

当有按键按下时，输出按键值。

```
void app_ps2_run(void)
```

```
{
    static u32 time_count = 0;
    if (millis() - time_count > 50) /* 50ms 检测一次 */
    {
        time_count = millis();
        ps2_write_read();          /* 读取ps2数据 */
        if (ps2_data.isConnected) /* 判断ps2是否连接 */
        {
            if (ps2_data.mode)
            {
                if (ps2_data.left_x != ps2_data.last_left_x)
                {
                    printf("\r\n left_x:%d\t", ps2_data.left_x);
                }
                if (ps2_data.left_y != ps2_data.last_left_y)
                {
                    printf("\r\n left_y:%d\t", ps2_data.left_y);
                }
                if (ps2_data.right_x != ps2_data.last_right_x)
                {
                    printf("\r\n right_x:%d\t", ps2_data.right_x);
                }
                if (ps2_data.right_y != ps2_data.last_right_y)
                {
                    printf("\r\n right_y:%d\t", ps2_data.right_y);
                }
            }
            if (ps2_data.left_up)
```

```
            {
                printf("PS2_LEFT_UP is pressed!\r\n");
            }
            else if (ps2_data.left_right)
            {
                printf("PS2_LEFT_RIGHT is pressed!\r\n");
            }
            else if (ps2_data.left_down)
            {
                printf("PS2_LEFT_DOWN is pressed!\r\n");
            }
            else if (ps2_data.left_left)
            {
                printf("PS2_LEFT_LEFT is pressed!\r\n");
            }
            else if (ps2_data.select)
            {
                printf("PS2_SELECT is pressed!\r\n");
            }
            else if (ps2_data.start)
            {
                printf("PS2_START is pressed!\r\n");
            }
            else if (ps2_data.right_up)
            {
                printf("PS2_RIGHT_UP is pressed!\r\n");
            }
            else if (ps2_data.right_right)
```

```
        {
            printf("PS2_RIGHT_RIGHT is pressed!\r\n");
        }
        else if (ps2_data.right_down)
        {
            printf("PS2_RIGHT_DOWN is pressed!\r\n");
        }
        else if (ps2_data.right_left)
        {
            printf("PS2_RIGHT_LEFT is pressed!\r\n");
        }
        else if (ps2_data.left_2)
        {
            printf("PS2_LEFT_2 is pressed!\r\n");
        }
        else if (ps2_data.right_2)
        {
            printf("PS2_RIGHT_2 is pressed!\r\n");
        }
        else if (ps2_data.left_1)
        {
            printf("PS2_LEFT_1 is pressed!\r\n");
        }
        else if (ps2_data.right_1)
        {
            printf("PS2_RIGHT_1 is pressed!\r\n");
        }
    }
```

```
    }

}
```

3. Arduino 例程说明

本小节讲解通过 Arduino 连接手柄的使用，以 Arduino UNO 为例，采用 PS2X_lib 库。

3.1. 硬件连接

例程硬件连接关系如下。

DATA——12

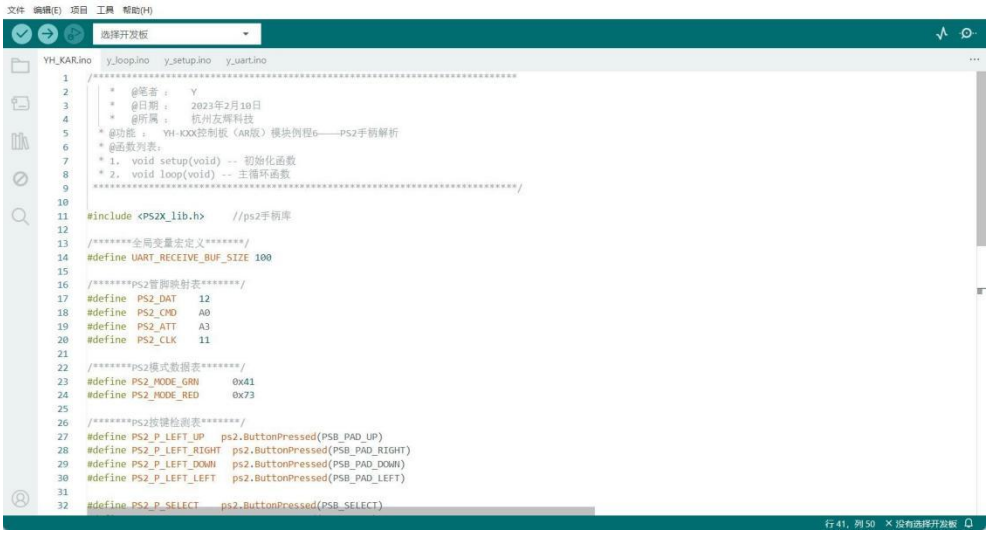
CMD——A0

CS——A3

CLK——11

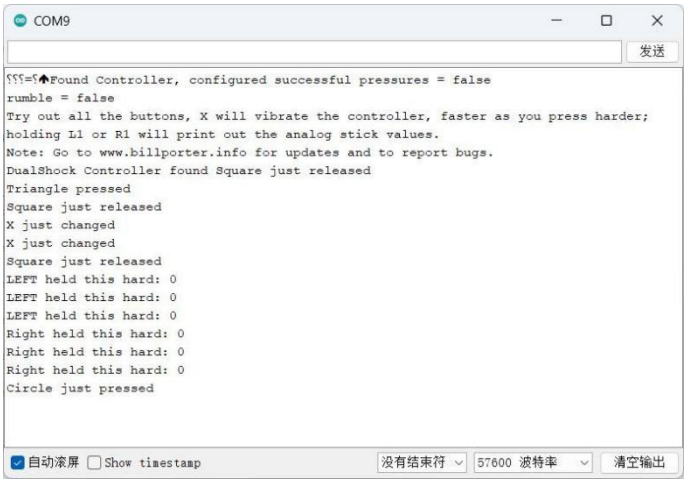
3.2. 程序说明

具体程序源码，请参考源码文件。PS2X 库的使用网上有非常多教程，这里不再赘述。

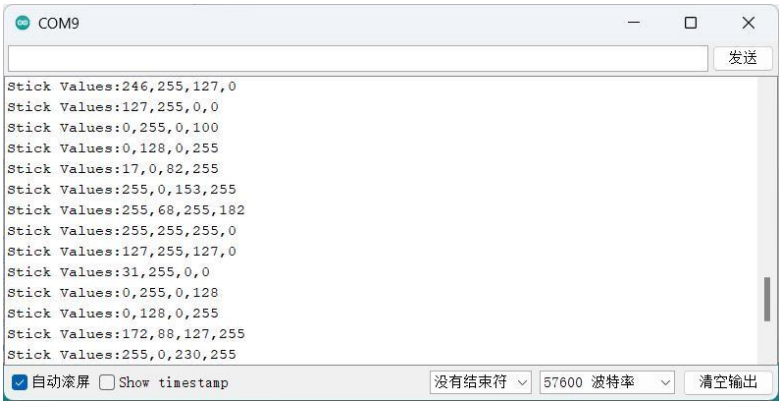


程序下载到 Arduino 后，打开对应串口，就可以参考 PS2 手柄数据的输出，按键就会有对应

键值显示



注意使用摇杆，需要按住 R1 或 L1 按键。波动摇杆，可以观察到数据变化。



4. 51 单片机例程说明

详见程序源码

5.FAQ 问题

如何判断 PS2 手柄与接收器连接成功？

给 PS2 和接收器供电，当你的 PS2 的指示灯，不再频闪开始常亮 ，说明你的 PS2 手柄与接收器连接成功。

锁定款接收器如何与 PS2 手柄连接？

按下接收器背部小孔中的按钮，接收器红灯开始快速闪烁，打开手柄开关，红灯不再闪烁说明手柄与接收器锁定成功，锁定款手柄出厂默认已与配套手柄锁定。

关于 PS2 手柄红绿灯模式切换？

PS2 手柄在 STM32 例程中，通过 MODE 按键可以调节模式，这里建议调节到红灯和绿灯都亮起的红绿灯模式再使用。Arduino 例程 MODE 按键无效。